

Herramientas de inteligencia artificial utilizadas

Proyecto **JIDOKA** — Auditoría de facturas de siniestros automotrices (Reto 2) · Equipo AdwenTech · Septiembre 2026

Producto presentado: aplicación web fullstack (Next.js 16 + TypeScript + Prisma/SQLite) que audita facturas de talleres contra el tarifario pactado, aplica regla de parada estilo jidoka, redacta un informe de discrepancias línea por línea con evidencia citada y escala a revisión humana. **Demo pública:** <https://jidoka.adwentech.com> · **Código:** github.com/jefferygyamerah/jidoka-auditor (repositorio público).

Nota de alcance: este documento declara herramientas de IA; las herramientas convencionales (Git, Node.js, Prisma ORM, Vivliostyle para este PDF, etc.) se mencionan solo como contexto.

1. Resumen de herramientas de IA

HERRAMIENTA	ETAPA	PROPÓSITO DE APLICACIÓN	RESULTADO / EVIDENCIA
Z.ai Super Z (agente de desarrollo)	Construcción inicial del prototipo	Generación guiada de la app completa: esquema de datos, motor de reglas, API, UI de 4 roles, semilla realista, verificación en navegador.	Prototipo funcional «JIDOKA» en un ciclo de trabajo; verificado en navegador por el propio agente (worklog del proyecto, tareas 1–2).
GLM (Z.ai) como modelo de agente en Hermes	Refactor de alineación, aseguramiento de calidad y preparación de la entrega	Reingeniería del prototipo hacia el flujo acordado: informe sin veredictos de pago, regla de parada, revisión por hallazgo, guardia anti-duplicados con contexto, evidencia citada; verificación end-to-end vía API. Después: pase de experiencia de usuario (una decisión por pantalla, lenguaje plano) y preparación del repositorio público con README para jueces.	Commits auditable (p. ej. <i>Alineación al flujo acordado #44, UX pass Apple</i>); typecheck limpio, build de producción OK, escenarios verificados; repositorio publicado sin claves ni datos reales.
z-ai-web-dev-sdk (LLM embebido en el producto)	Funcionalidad del producto (informe del agente)	Redacta el informe de auditoría línea por línea a partir de hallazgos ya demostrados por reglas deterministas; no decide pagos (guard <code>JIDOKA_DISABLE_IA</code> y ruta de respaldo determinista).	34 informes generados en la semilla de demostración, consultables en la demo pública; el diseño del producto exige que el LLM solo redacte lo que el motor determinista ya probó con cita de evidencia.
Claude (Anthropic) y ChatGPT	Dirección de producto y control	Sesiones de dirección con el fundador para fijar alcance, arquitectura y reglas del flujo de auditoría antes de construir; revisión crítica de límites del agente.	Decisiones registradas en el sistema de gestión (Leantime #44): informe-only, parada con montos «sin evaluar», revisión por hallazgo; el alcance quedó definido antes del refactor.

HERRAMIENTA	ETAPA	PROPÓSITO DE APLICACIÓN	RESULTADO / EVIDENCIA
Síntesis de voz (edge-TTS) orquestada por Hermes	Material del concurso (video de demostración)	Producción del video demo de 2:53: guion estructurado en 8 secciones (problema, valor, panel, hallazgos, agente, pruebas, cierre) y narración en español latino con movimientos de cámara sutiles.	Video entregado como material de difusión del equipo; control de calidad visual fotograma a fotograma (14 cuadros verificados) antes de la entrega.

2. El diseño del agente: núcleo determinista + humano en el circuito

El aporte metodológico del proyecto se resume en dos principios que gobiernan toda la arquitectura: **las decisiones de auditoría son deterministas** (reglas verificables sobre evidencia, mismo input → mismo resultado, cero montos inventados por IA) y **toda decisión de negocio queda en manos de una persona** (el agente informa, nunca aprueba pagos). Incluso la salida del LLM pasa por una **validación determinista** antes de publicarse: si falla, se emite el informe determinista. Los dos diagramas siguientes muestran ambos principios tal como operan en el producto.

El núcleo determinista: el motor decide — el LLM solo explica
Toda conclusión de auditoría nace de reglas verificables; la IA generativa redacta, nunca decide.

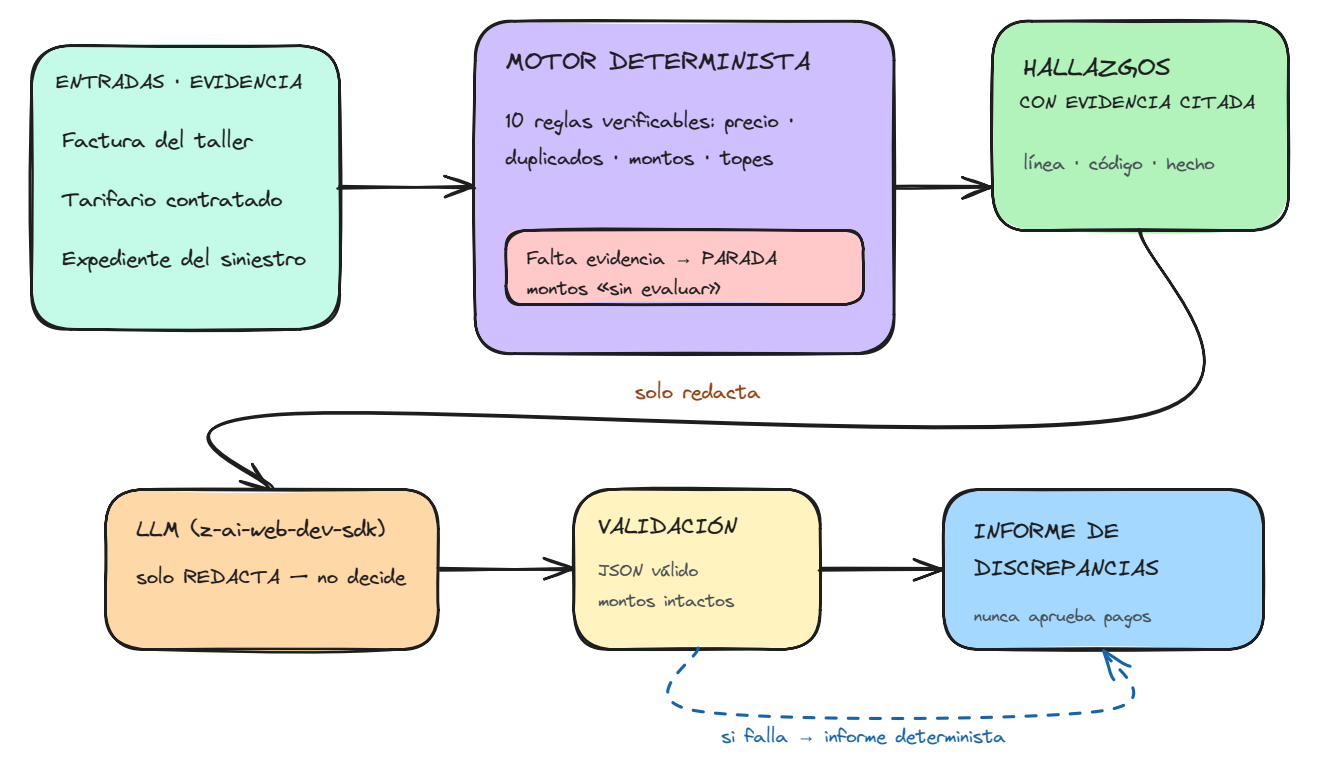


Figura 1 · El núcleo determinista. El motor de reglas examina factura, tarifario y expediente; cada hallazgo sale con evidencia citada. El LLM (z-ai-web-dev-sdk) solo redacta el informe y su salida pasa por una compuerta de validación (JSON válido, montos intactos, aviso «sin evaluar» preservado); si falla, se emite el informe determinista (bandera JIDOKA_DISABLE_IA).

Humano en el circuito: el agente informa, el humano decide

Cada hallazgo se acepta, se pide evidencia o se descarta con motivo — la decisión nunca es del agente.

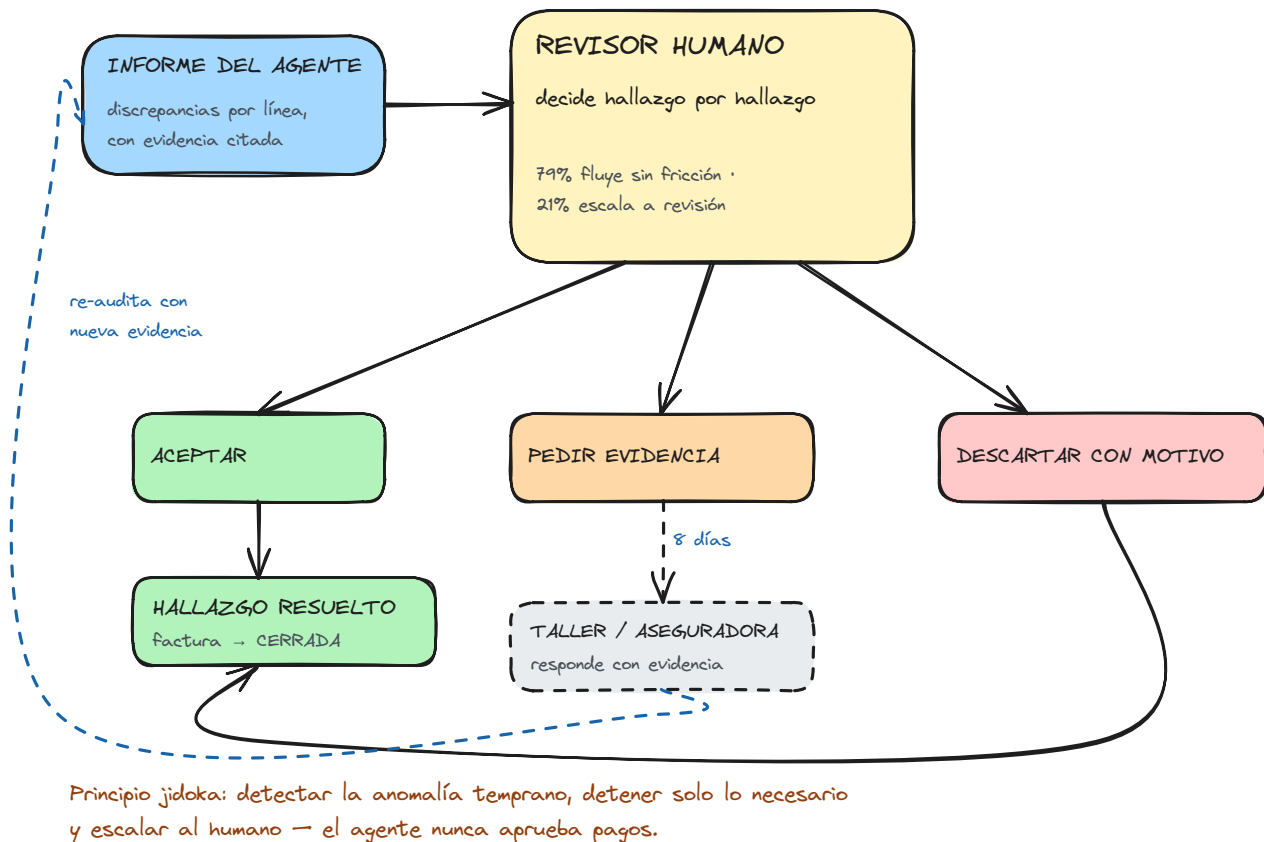


Figura 2 · Humano en el circuito. El revisor decide hallazgo por hallazgo: aceptar, pedir evidencia o descartar con motivo. La evidencia nueva regresa al agente para re-auditar; el cierre de la factura ocurre solo tras revisión humana.

3. Aplicación por etapa del proyecto

Etapa 1 — Concepto y especificación (antes de escribir código)

El alcance se definió aplicando principios del Toyota Production System al proceso de auditoría: definir primero el flujo manual ideal, detectar la anomalía temprano, detener solo lo necesario y reservar al humano el juicio. Con Claude/ChatGPT como interlocutores de dirección se fijaron las reglas del flujo acordado: el agente produce un **informe de discrepancias** y nunca aprueba ni rechaza pagos; las líneas sin evidencia compartida quedan «sin evaluar»; cada hallazgo es aceptado, se le pide evidencia o se descarta con motivo.

Etapa 2 — Construcción del prototipo (Z.ai Super Z)

Super Z construyó el prototipo inicial completo: modelo de datos (Taller, TarifarioItem, Siniestro, Factura, Partida, Hallazgo, LogAgente, Revision), motor determinista de reglas, API, interfaz multi-rol y datos de demostración panameños. El mismo agente verificó el resultado en navegador y corrigió defectos detectados (etiquetas del gráfico de Pareto, campos faltantes en la API).

Etapa 3 — Alineación al flujo acordado y QA (Hermes + GLM)

Un segundo agente (AgentKwame sobre Hermes, modelo GLM de Z.ai) ejecutó la reingeniería con puertas de verificación: comparación del prototipo contra la especificación acordada, refactor de motor, API y UI, datos de prueba que demuestran cada regla, y verificación end-to-end vía API (precio fuera de tarifario; duplicado real;

repetición legítima con contexto documentado que no debe marcarse; parada por falta de tarifario con montos «sin evaluar»; revisión por hallazgo persistida y releída).

Etapas 4 — IA dentro del producto (z-ai-web-dev-sdk)

Dentro de la aplicación, el LLM redacta el informe del revisor a partir de hallazgos deterministas ya citados con evidencia. La separación es deliberada y es el aporte metodológico del proyecto: **el motor determinista decide qué señalar; el LLM solo explica**. Sin LLM, la aplicación sigue operando con respaldo determinista (bandera de entorno para pruebas reproducibles).

Etapas 5 — Endurecimiento y publicación (equipo humano + agentes)

Un ingeniero del equipo aportó vía revisión por pares (PR #1) la suite de **fixtures con resultado esperado por expediente** y el comando `doctor`, que valida entorno y esquema completo sin claves de nube — la ausencia de claves es parte del contrato del producto. En paralelo, el agente en Hermes ejecutó el pase de experiencia de usuario y la preparación del repositorio público: retiro de artefactos locales, semilla regenerable (`db/seed-db.sql`), `.env.example` y un README para jueces con guion de demostración de 2 minutos. El repositorio se publicó y la demo quedó accesible en <https://jidoka.adwentech.com> (verificada: respuesta HTTP 200 con las 34 facturas sintéticas de la semilla).

4. Resultados medibles del uso de IA

- **Tiempo concepto → prototipo → alineación → publicación:** días, no semanas, con historial de commits y el PR #1 como evidencia verificable del recorrido completo.
- **Cobertura de auditoría:** 100% de las facturas auditadas al instante; el 79% fluye sin revisión humana obligatoria (27 de 34 en flujo o revisión estándar, 7 cerradas sin pendientes). En la demo pública en vivo: 18 hallazgos detectados, 9 ya resueltos por revisión humana.
- **Precisión del alcance:** 18 pruebas automáticas (bun) incluyendo los fixtures con resultado esperado por expediente; typecheck y build de producción limpios en la entrega.
- **Control de alucinación:** el LLM de producto opera sobre hallazgos deterministas con cita obligatoria de evidencia (línea de factura, código de tarifario, hecho del siniestro), con prueba reproducible de que no inventa montos.
- **Verificabilidad pública:** demo en vivo, repositorio público clonable y semilla SQL regenerable — cualquier evaluador puede reproducir el estado de demostración sin credenciales de nube.

5. Declaración de responsabilidad

El equipo revisó y aprobó cada artefacto entregado; las herramientas de IA actuaron bajo dirección humana y sus salidas fueron verificadas (compilación, pruebas automáticas y revisión humana — incluida la revisión por pares del PR #1) antes de su inclusión. Los datos del prototipo y de la demo pública son sintéticos y no contienen información real de aseguradoras, talleres ni personas.

Documento generado con Vivliostyle a partir de fuente HTML editable. Repositorio del proyecto: github.com/jefferygyamerah/jidoka-auditor (público) · Demo en vivo: jidoka.adwentech.com.